CHENNAI SMART TRAFFIC MANAGEMENT USING A* SEARCH

LLT Mini-Project Submission

Course: Fundamentals of Artificial Intelligence

Student: Yusuf H.

Register Number: RA2612049015077

Platform: Google Colab

Submitted as part of the Fundamentals of Artificial Intelligence course.

1. Project Overview

This project applies the A* search algorithm to find a route through Chennai's drivable road network from Perambur – Vyasarpadi to Tambaram Bypass. It compares estimated route travel time under three traffic scenarios: Normal, Medium and Heavy. The road network is represented as a graph, and each scenario uses a different edge travel-time weight.

2. Problem Statement

A route that is suitable under normal traffic may not be the best option when traffic speeds decrease. This project models the road network and traffic observations, then uses heuristic search to identify a low-cost path for each scenario. The cost being optimized is estimated travel time, not physical distance alone.

3. Aim and Objectives

• Represent Chennai's drivable roads as a graph using OpenStreetMap data through OSMnx.
• Load the Normal, Medium and Heavy traffic CSV datasets.
• Match traffic observation coordinates to nearby road nodes.
• Derive scenario-specific estimated travel-time costs for road edges.
• Use A* search with a straight-line-distance heuristic to calculate routes.
• Compare route node counts and estimated travel times, and plot the routes.

4. Journey Details

Field	Value
Source	Perambur – Vyasarpadi
Source coordinates	13.114385, 80.238720
Destination	Tambaram Bypass
Destination coordinates	12.924996, 80.099676

5. Tools and Libraries

• Google Colab
• Python
• OSMnx for the OpenStreetMap road network, nearest-node lookup, distance calculations and route plotting
• NetworkX for A* search using *nx.astar_path()*
• Pandas for traffic CSV processing and result export
• NumPy and SciPy cKDTree for nearest traffic-node mapping
• Matplotlib for route visualization

6. Data Inputs

The project uses three traffic CSV files: *chennai_traffic_dataset.csv*, *chennai_traffic_MEDIUM.csv* and *chennai_traffic_HEAVY.csv*. The code uses location and speed information and inspects traffic density. The road graph is requested for Chennai using OSMnx with the driving network type.

7. A* Search and Heuristic

Search algorithm: The project uses NetworkX's A* search for Normal, Medium and Heavy traffic, with separate edge weights for each scenario.

Heuristic: The heuristic calculates the great-circle distance between the current node and the goal node, then divides it by an assumed speed of 120 km/h converted to metres per second. This estimates the remaining travel time in seconds.

```
h(n) = great-circle distance(n, goal) ÷ (120 / 3.6)
```

Path cost: A* uses estimated travel time as the path cost. The edge weights are calculated separately for each traffic scenario.

8. Traffic Cost Calculation

Traffic observations are mapped to nearby road nodes. The graph is projected into metres and a cKDTree is used to associate each road node with the nearest observed traffic node. For each scenario, edge travel time is calculated from the edge length and the average speed at its endpoint nodes. A minimum speed of 5 km/h is used to avoid zero or very small speeds.

9. Implementation Workflow

1. Install and import the required libraries in Google Colab.
2. Request Chennai's drivable road graph through OSMnx.
3. Load the three traffic CSV files and inspect their fields and speed values.
4. Map traffic observations to nearest road nodes.
5. Calculate travel-time edge weights for each traffic scenario.
6. Find the graph nodes nearest to the source and destination coordinates.
7. Define the heuristic and run A* for Normal, Medium and Heavy traffic.
8. Calculate route node counts and estimated travel times.
9. Plot the routes and export the comparison results.

10. Outputs

• Comparison table for traffic condition, route node count and estimated travel time.
• Combined route visualization for Normal, Medium and Heavy traffic.
• Individual route images for each scenario.
• Results exported to *chennai_astar_results.csv*.
• A recommendation based on the lowest estimated route travel time.

11. Results and Interpretation

The saved results from the A* runs in the notebook are shown below.

Traffic condition	Route nodes	Estimated time
Normal	450	21.38 minutes
Medium	381	18.81 minutes
Heavy	376	32.23 minutes

Interpretation: The Medium traffic scenario has the lowest estimated travel time at 18.81 minutes. The Heavy traffic scenario has the highest estimated travel time at 32.23 minutes. Route node count indicates how many graph nodes are included in the path; it does not directly represent distance or execution time.

12. Assumptions and Limitations

• Travel times are estimates based on the provided traffic observations and road-edge lengths; they are not guaranteed live navigation times.
• Results depend on the traffic CSV data and the road graph available during execution.
• The heuristic uses straight-line distance and an assumed speed of 120 km/h; its admissibility for every edge depends on the speed assumptions.
• The nearest-node mapping depends on the coverage and location accuracy of the traffic observations.
• OpenStreetMap graph retrieval requires internet access and may be affected by Overpass service errors.

13. Conclusion

This project demonstrates heuristic search using A* on a road-network graph. By changing the estimated travel-time edge weights for Normal, Medium and Heavy traffic, the project compares the resulting routes and estimated travel times for the same source and destination. In the saved results, the Medium traffic scenario has the lowest estimated travel time.